

Advanced Machine Learning

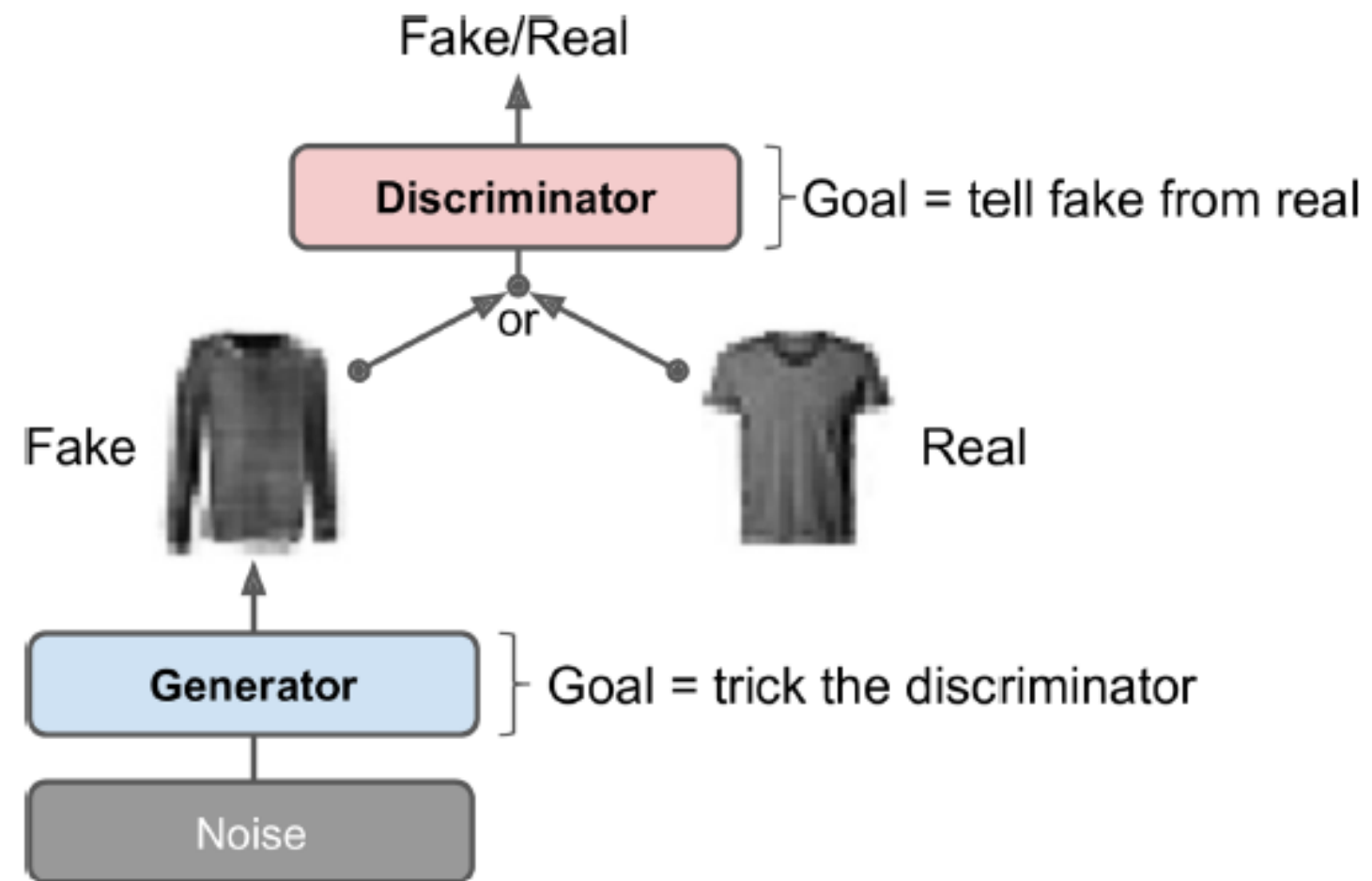
DATA 642

Lecture 15 — Introduction to Generative Adversarial Neural Networks (GANs)

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Generative Adversarial Networks

- Generative adversarial networks were proposed in a 2014 paper by Ian Goodfellow et al., and although the idea got researchers excited almost instantly, it took a few years to overcome some of the difficulties of training GANs.
- A GAN is composed of two neural networks.



Applications of GANs

1. **Image Generation and Editing:** GANs can generate high-resolution, realistic images of faces, objects, landscapes, and more. These generated images can be used in various applications such as art generation, content creation, and even in the fashion industry for generating new designs. Additionally, GANs can be used for image editing tasks like inpainting missing regions in images.
2. **Data Augmentation:** GANs can be used to augment training data for machine learning models. By generating synthetic data that closely resembles the real data, GANs can help in expanding limited datasets, which is particularly useful in domains where collecting large amounts of labeled data is expensive or time-consuming.
3. **Text-to-Image Synthesis:** GANs can be used to generate realistic images from textual descriptions. Given a textual description, such as a sentence or a paragraph, GANs can generate images that correspond to the described scene or object. This has applications in generating artwork from textual prompts, enhancing virtual reality experiences.
4. **Style Transfer:** GANs can be employed for transferring styles between images. By learning the style of one image and applying it to another image, GANs can generate visually appealing images with a desired artistic style. This technique has applications in photo editing software, where users can apply various artistic styles to their photos, as well as in the creation of visual effects for movies and games.

5. **Molecular Generation:** GANs can be used to generate molecular structures with desirable properties, such as new drug candidates or materials with specific characteristics. By training on a dataset of known molecular structures and properties, GANs can learn to generate novel molecules that exhibit desired properties. This has applications in drug discovery, material science, and computational chemistry.
6. **Anomaly Detection:** GANs can be employed for anomaly detection in various types of data, such as images, time series, or tabular data. By training a GAN on normal data samples, anomalies or outliers can be detected as instances that deviate significantly from the learned distribution. This has applications in cybersecurity, fraud detection, quality control, and predictive maintenance.
7. **Domain Adaptation and Style Transfer:** GANs can be used for domain adaptation, where the goal is to transfer knowledge learned from a source domain to a target domain with different characteristics. By learning to map samples from one domain to another while preserving their semantic content, GANs can adapt models trained on one domain to perform well on a different domain. This has applications in computer vision, natural language processing, and other domains where labeled data in the target domain is scarce.

These are just a few examples of the wide range of applications of GANs across different domains. GANs continue to inspire innovative research and practical applications in various fields.

Generator

- Takes a random distribution as input (typically Gaussian) and outputs some data, for instance an image.
- We can think of the random inputs as the latent representations (i.e., codings) of the image to be generated. The generator offers the same functionality as a decoder in a variational autoencoder, and it can be used in the same way to generate new images (just feed it some Gaussian noise, and it outputs a brand-new image). However, it is trained very differently.
- To mathematically formulate this, the generator transforms the input noise vector, $\mathbf{z}$, into a sample $\mathbf{x} = G(\mathbf{z}; \boldsymbol{\theta}_g)$ of the same dimension as that of the available observations, $\mathbf{x}_n$, $n = 1, \dots, N$. The parameter vector $\boldsymbol{\theta}_g$ comprises all the parameters that define the generator neural network and these parameters have to be estimated during the training phase. The goal of the training is to compute the parameters so that the generated patterns, $\mathbf{x}$, are statistically indistinguishable from the available observations. From now on, we will refer to the observations as real and those that are produced by the generator as fake data.

Discriminator

- Takes either a fake image from the generator or a real image from the training set as input, and must guess whether the input image is fake or real. The discriminator is a binary classifier that is implemented as a deep neural network. Its input is fed with samples $\mathbf{x}$ and it outputs a corresponding probability value, $y = D(\mathbf{x}; \boldsymbol{\theta}_d)$. The parameter vector $\boldsymbol{\theta}_d$ comprises all the parameters that define the associated neural network and are learned during training. The output value, y , represents the probability that the corresponding input pattern, $\mathbf{x}$, originates from the observations rather than from the generator. In other words, $D(\mathbf{x}; \boldsymbol{\theta}_d)$ is the probability of $\mathbf{x}$ being real, and $1 - D(\mathbf{x}; \boldsymbol{\theta}_d)$ is the probability of $\mathbf{x}$ being fake.

Training

- During training, the generator and the discriminator have opposite goals: the discriminator tries to tell fake images from real images, while the generator tries to produce images that look real enough to trick the discriminator. Because the GAN is composed of two networks with different objectives, it cannot be trained like a regular neural network. Each training iteration is divided into two phases.
 - In the first phase, we train the discriminator. A batch of real images is sampled from the training set and is completed with an equal number of fake images produced by the generator. The labels are set to 0 for fake images and 1 for real images, and the discriminator is trained on this labeled batch for one step, using the binary cross-entropy loss. Importantly, backpropagation only optimizes the weights of the discriminator during this phase.
 - In the second phase, we train the generator. We first use it to produce another batch of fake images, and once again the discriminator is used to tell whether the images are fake or real. This time we do not add real images in the batch, and all the labels are set to 1 (real): in other words, we want the generator to produce images that the discriminator will (wrongly) believe to be real! Crucially, the weights of the discriminator are frozen during this step, so backpropagation only affects the weights of the generator.

The Cost Function

During training, the parameter vectors are optimally estimated so that the discriminator confuses the real with the fake data points. The following two-player min-max game value (cost) is adopted:

$$\min_{\boldsymbol{\theta}_g} \max_{\boldsymbol{\theta}_d} J(\boldsymbol{\theta}_g, \boldsymbol{\theta}_d), \quad (1)$$

where

$$J(\boldsymbol{\theta}_g, \boldsymbol{\theta}_d) = E_{\mathbf{x} \sim p_r(\mathbf{x})} [\ln D(\mathbf{x}; \boldsymbol{\theta}_d)] + E_{\mathbf{z} \sim p_z(\mathbf{z})} [\ln(1 - D(G(\mathbf{z}; \boldsymbol{\theta}_g); \boldsymbol{\theta}_d))] , \quad (2)$$

and $p_r(\mathbf{x})$ denotes the probability distribution associated with the real data (subscript r reminds us of real). In words, if $\boldsymbol{\theta}_g$ is kept fixed, then the discriminator is trained, via $\boldsymbol{\theta}_d$, so that its output is maximized both for real $D(\mathbf{x}; \boldsymbol{\theta}_d)$ and for fake $1 - D(G(\mathbf{z}; \boldsymbol{\theta}_g); \boldsymbol{\theta}_d)$, in order to confuse the discriminator.

Algorithm

- Initialize $\boldsymbol{\theta}_d^{(0)}$ and $\boldsymbol{\theta}_g^{(0)}$.
- Set the minibatch size equal to K .
- Set the number of iterations, m , for the discriminator; the simplest case is $m = 1$, and it is used in the original paper.
- **While** $\boldsymbol{\theta}_d$ and $\boldsymbol{\theta}_g$ have not converged, **Do**

- For $t = 1, \dots, m$, **Do**

- Sample $\mathbf{z}^{(i)}$, $i = 1, \dots, K$, from $p_{\mathbf{z}}(\mathbf{z})$
- Sample $\mathbf{x}^{(i)}$, $i = 1, \dots, K$, from $p_r(\mathbf{x})$
- Compute the gradient

$$\nabla_{\boldsymbol{\theta}_d} \left\{ \frac{1}{K} \sum_{i=1}^K (\ln D(\mathbf{x}^{(i)}; \boldsymbol{\theta}_d) + \ln(1 - D(G(\mathbf{z}^{(i)}; \boldsymbol{\theta}_g); \boldsymbol{\theta}_d))) \right\} \quad (3)$$

- Update $\boldsymbol{\theta}_d$ via a gradient ascent scheme

- **End For**

Algorithm (cont.)

- Sample $\mathbf{z}^{(i)}$, $i = 1, \dots, K$, from $p_{\mathbf{z}}(\mathbf{z})$
- Compute the gradient

$$\nabla_{\boldsymbol{\theta}_g} \left\{ \frac{1}{K} \sum_{i=1}^K \ln(1 - D(G(\mathbf{z}^{(i)}; \boldsymbol{\theta}_g); \boldsymbol{\theta}_d)) \right\} \quad (4)$$

- Update $\boldsymbol{\theta}_g$ via a gradient descent scheme
- **End While**